

Complete Architecture and Implementation Plan

A practical seven-step blueprint for building the complete frontend intern assignment: canonical state, responsive viewport isolation, safe edit commands, canvas and code synchronization, deterministic AI proposals, independent recovery, testing, documentation, and deployment.

The core idea
A change is safe only when the target element IDs, editable fields, viewport scope, and revision are valid. Nothing produced by the AI demo changes the template until the user accepts it.

The complete seven-step route

Step	Build area	Outcome
1	Foundation and architecture	Repository, editor shell, state ownership, folder structure
2	Canonical model and responsive rendering	Stable IDs, JSON model, viewport overrides, renderer
3	Command, validation, revision, and history core	One safe pipeline for every committed change
4	Canvas selection and manual editing	Single/group selection, inspector, inline editing, structure
5	Code surface and deterministic AI proposals	Validated JSON editing, proposal review, partial acceptance
6	Persistence, accessibility, and verification	Refresh safety, keyboard support, focused automated tests
7	Documentation, deployment, and submission	README, PRODUCT_NOTES, AI_USAGE, Vercel, final demo

Prepared for: Chetan Sai Vatti
Project type: React + TypeScript frontend hiring assignment
Recommended persistence: Browser localStorage; no backend or real AI model required

Project interpretation

This is a browser-based website-builder prototype around one responsive, one-page template. The difficult part is not the page design. The difficult part is proving that every edit is limited to the user's selected elements and chosen viewport, that proposed AI changes are reviewed before application, and that one element can be restored without rolling back unrelated work.

The reviewer must be able to complete this journey

- Load one original or appropriately licensed responsive template.
- Switch between desktop, tablet, and mobile previews.
- Select one element and build a group using Shift/Ctrl/Cmd-click or marquee selection.
- Edit content, style, size, position, order, and structure through the canvas or code surface.
- Choose All views or a single Desktop, Tablet, or Mobile edit scope.
- Enter a documented instruction into a deterministic, predefined AI scenario engine.
- Review a before/after proposal for every returned element without changing the template yet.
- Accept or reject each proposed element independently.
- Restore one earlier element revision at one viewport scope without changing other work.
- Refresh and retain the current template and history; deliberately reset when requested.

What not to build

Avoid	Reason
Real LLM integration	The assignment explicitly requests a deterministic scenario engine.
Backend or database	localStorage is sufficient for refresh persistence and keeps the scope frontend-focused.
Authentication	There is one local editor user; accounts add no required evidence.
Multiple templates	One polished responsive template is required and easier to verify deeply.
Raw AI-generated HTML	The durable source of truth must remain a typed JSON-serializable model.
Global-only undo	Recovery must be per element and per viewport scope.

Definition of a safe completed edit

The user deliberately chooses the targets and scope, reviews any generated proposal, commits through validation, sees the intended viewport update, receives a scoped history entry, and can later restore that element without affecting unrelated elements or views.

STEP 1

Foundation and full application architecture

Objective: Create the repository and define which layer owns every responsibility before building visual editing.

Deliverable: A running React TypeScript application, architecture boundaries, state slices, original template plan, and documentation skeleton.

1.1 Recommended stack

Technology	Responsibility
React + TypeScript + Vite	Editor UI, template renderer, type-safe components
Redux Toolkit	Shared canonical state used by canvas, code, AI, history, and viewport controls
Zod	Runtime validation of commands, saved data, code input, and AI proposals
CodeMirror	Polished JSON editing surface with error feedback
dnd-kit	Pointer and keyboard-capable reordering
Vitest + Testing Library	Unit and component tests for safety behavior
Playwright	A few complete reviewer journeys in a real browser
localStorage	Current template and history persistence
Vercel	Static deployment

1.2 System ownership



Layer	Owns
Editor UI	Captures user intent and maintains drafts; never directly mutates template values
Command creators	Translate canvas, code, AI acceptance, and restore actions into typed commands
Command executor	Runs schema, authority, field, scope, revision, and structure validation
Canonical state	Stores normalized elements, base values, viewport overrides, and revisions
Viewport resolver	Merges base values with the active viewport override
Renderer	Renders normal React components from resolved JSON values
History	Stores before/after snapshots per element and scope
Persistence	Saves only validated durable state after successful commits

1.3 Editor screen architecture

TOP TOOLBAR		
Layers and elements	Responsive website canvas	Design / AI / Code / History
Stable IDs Single/group selection Add, duplicate, hide Move and reorder	Desktop 1440 Tablet 768 Mobile 375 Selection overlays Inline editing and handles	Inspector controls Instruction examples Validated JSON draft Scoped revision list
PROPOSAL DRAWER: Before / After / Changed fields / Accept / Reject / Status		

1.4 Redux state boundaries

```
interface RootState {
  template: TemplateState; // durable website state
  history: HistoryState; // durable scoped revisions
  editorUI: EditorUIState; // selection, preview, scope, open panel
  proposals: ProposalState; // temporary pending/accepted/rejected results
}
```

- Persist template and history. Selection, open panels, drafts, and pending proposals may remain temporary.
- Keep activeViewport and editScope separate: one controls what the user sees; the other controls what changes.
- Use one validated Redux action for a committed change so template updates and history creation are atomic.

1.5 Repository setup

```
npm create vite@latest scoped-ai-template-editor -- --template react-ts
cd scoped-ai-template-editor
npm install
npm install @reduxjs/toolkit react-redux zod
npm install @uiw/react-codemirror @codemirror/lang-json
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
npm install -D vitest jsdom @testing-library/react
npm install -D @testing-library/jest-dom @testing-library/user-event
npm install -D @playwright/test
```

Start evidence collection now

Create README.md, PRODUCT_NOTES.md, and AI_USAGE.md on the first day. Record short redacted AI interactions and decisions as they happen instead of trying to reconstruct them before submission.

1.6 Complete folder architecture

```
src/
  app/
    store.ts
    hooks.ts
    persistenceMiddleware.ts
  model/
    types.ts
    schemas.ts
    editableFields.ts
    seedTemplate.ts
    resolveElement.ts
    resolveChildren.ts
  commands/
    commandTypes.ts
    commandSchemas.ts
    commandExecutor.ts
    validateCommand.ts
    validateStructure.ts
    createRevisionToken.ts
  store/
    templateSlice.ts
    editorUISlice.ts
    proposalSlice.ts
    historySlice.ts
    selectors.ts
  editor/
    EditorShell.tsx
    TopToolbar.tsx
    LayersPanel.tsx
    Canvas.tsx
    DeviceFrame.tsx
    SelectionOverlay.tsx
    DesignInspector.tsx
    CodePanel.tsx
    HistoryPanel.tsx
  renderer/
    TemplateRenderer.tsx
    ElementRenderer.tsx
  elements/
    SectionElement.tsx
    ContainerElement.tsx
    TextElement.tsx
    ButtonElement.tsx
    ImageElement.tsx
  ai/
    types.ts
    schemas.ts
    scenarioEngine.ts
    scenarios/
      contentScenario.ts
      styleScenario.ts
      resizeScenario.ts
      responsiveScenario.ts
      compactGroupScenario.ts
    AIInstructionPanel.tsx
    ProposalDrawer.tsx
  history/
    types.ts
    createHistoryEntry.ts
    createRestoreCommand.ts
  persistence/
    storage.ts
    persistedStateSchema.ts
  tests/
    responsiveIsolation.test.ts
    commandValidation.test.ts
    aiSelectionScope.test.ts
    staleRevision.test.ts
    canvasCodeConsistency.test.ts
    independentRestore.test.ts
  App.tsx
  main.tsx
  README.md
  PRODUCT_NOTES.md
  AI_USAGE.md
```

Step 1 completion checklist

- React TypeScript application runs locally.
- Redux store contains four clear slices.
- The editor shell has toolbar, layer panel, center canvas, right panel, and proposal drawer placeholders.
- All architecture folders and three required documentation files exist.
- An original one-page business template has been chosen and documented.

STEP 2

Canonical JSON model and responsive template rendering

Objective: Build the durable source of truth with stable element IDs and explicit viewport overrides.

Deliverable: One complete template rendered from JSON at desktop, tablet, and mobile widths with proven viewport isolation.

2.1 Original one-page template

Use an original small-business landing page to avoid licensing complexity. A digital studio, coffee shop, or consulting company works well. The template should be visually credible but intentionally limited to element types that your model can safely edit.

Area	Stable IDs
Page	page.root
Navigation	nav.root, nav.logo, nav.links, nav.cta
Hero	hero.section, hero.content, hero.heading, hero.description, hero.primaryButton, hero.visual
Services	services.section, services.heading, services.grid, services.card.1, services.card.2, services.card.3
Testimonial	testimonial.section, testimonial.quote, testimonial.author
Call to action	cta.section, cta.heading, cta.description, cta.button
Footer	footer.root, footer.brand, footer.copyright

Stable selection authority

Selection must store these IDs. Do not infer identity from CSS classes, visible text, React tree position, or DOM order.

2.2 Normalized element model

```
type Viewport = "desktop" | "tablet" | "mobile";
type EditScope = "all" | Viewport;
type ElementType = "page" | "section" | "container" | "text" | "button" | "image";

interface ElementValues {
  content: ContentProperties;
  style: StyleProperties;
  layout: LayoutProperties;
}

interface TemplateElement {
  id: string;
  name: string;
  type: ElementType;
  base: ElementValues;
  overrides: Partial<Record<Viewport, DeepPartial<ElementValues>>>>;
  revisions: { base: number; desktop: number; tablet: number; mobile: number };
}

interface TemplateState {
  schemaVersion: number;
  templateId: string;
  templateName: string;
  rootElementId: string;
  elements: Record<string, TemplateElement>;
}
```

Store `parentId` and `order` on each element rather than maintaining a second `children` array. The renderer finds all resolved elements whose `parentId` matches the current container and sorts them by `order`. This avoids two sources of structural truth.

2.3 Editable property boundaries

Element	Allowed fields
Text	content.text; color; font size/weight; alignment; margin; width; order; visibility
Button	label; href; text/background color; padding; radius; width; order; visibility
Image	src; alt text; width; height; radius; order; visibility
Container	background; padding; margin; display; direction; gap; width; parent; order; visibility
Section	background; padding; display; direction; gap; width; order; visibility
Protected fields	id, type, revision counters, history, proposal status, schema version

2.4 Responsive resolution



```

function resolveElement(element: TemplateElement, viewport: Viewport): ElementValues {
  return deepMerge(element.base, element.overrides[viewport] ?? {});
}
  
```

Scope	Write location	Safety rule
All views	Write to element.base	Flows to every view except properties with an existing override
Desktop only	Write to overrides.desktop	Tablet and mobile remain byte-for-byte unchanged
Tablet only	Write to overrides.tablet	Desktop and mobile remain byte-for-byte unchanged
Mobile only	Write to overrides.mobile	Desktop and tablet remain byte-for-byte unchanged

Resolution example

```

base.style.fontSize = 52
overrides.mobile.style.fontSize = 34
  
```

```

Desktop result = 52
Tablet result  = 52
Mobile result  = 34
  
```

If base changes to 56, mobile remains 34 because the mobile override wins.

2.5 Renderer architecture

```

TemplateRenderer
-> gets rootElementId
-> ElementRenderer(root)
  -> resolveElement(element, activeViewport)
  -> resolveChildren(element.id, activeViewport)
  -> choose Section / Container / Text / Button / Image component
  -> recursively render resolved children
  
```

- Template CSS must come from resolved model values. Editor chrome CSS may use normal responsive styles.
- Soft deletion uses layout.visible = false, allowing viewport-specific hiding and recovery.
- Moving or reordering changes parentId and/or order at the chosen scope.

2.6 Device preview architecture

Preview	Intrinsic width	Implementation
Desktop	Approximately 1440 px	Scale the intrinsic canvas to fit the center area
Tablet	Approximately 768 px	Display within the same device-frame component
Mobile	Approximately 375 px	No horizontal overflow or clipped editor controls

The editor must remain usable at 1280 px overall. Render the preview at its intrinsic target width and calculate a scale factor from the available center area. Keep selection overlays inside the scaled canvas so their coordinates remain aligned.

2.7 Revision tokens

```
interface RevisionToken {
  base: number;
  viewport?: number;
}

// Mobile proposal records both current base and mobile revisions.
const token = {
  base: element.revisions.base,
  viewport: element.revisions.mobile,
};
```

A mobile proposal becomes stale when its element base or mobile revision changes. An unrelated desktop-only edit does not invalidate that mobile proposal. This is safer and less restrictive than one global revision number.

Step 2 completion checklist

- The complete page renders from the normalized JSON dictionary.
- Every editable element has a human-readable stable ID.
- Desktop, tablet, and mobile display correctly at their required widths.
- A mobile override changes only mobile; a base edit preserves existing overrides.
- Renderer logic contains no independent duplicate copy of template state.

STEP 3

Unified command, validation, revision, and history core

Objective: Make every committed manual, code, AI, and restore edit pass through one safe transaction boundary.

Deliverable: Typed commands, runtime validation, stale protection, atomic commits, scoped history, and restore-as-new-commit.

3.1 Typed edit command

```
type EditSource = "canvas" | "code" | "ai" | "restore";
```

```
interface PropertyChange {
  path: string;
  oldValue: unknown;
  newValue: unknown;
}
```

```
interface TargetEdit {
  elementId: string;
  revisionToken: RevisionToken;
  changes: PropertyChange[];
}
```

```
interface EditCommand {
  id: string;
  source: EditSource;
  scope: EditScope;
  selectedIdsSnapshot?: string[];
  targets: TargetEdit[];
  timestamp: string;
}
```

The selectedIdsSnapshot is mandatory for AI-originated commands. It records the authority that existed when the proposal was generated. Acceptance also checks the current selection according to the documented policy.

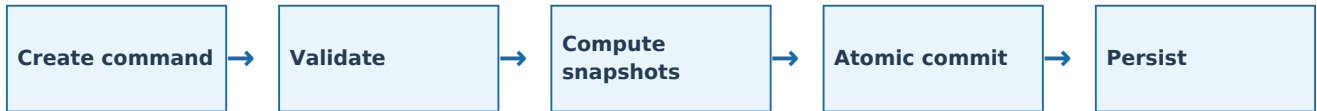
3.2 Validation order

Gate	Reject when
1. Schema	Command shape, scope, source, target structure, values, and unknown keys
2. Existence	Template, target element, and proposed parent IDs exist
3. Authority	AI targets are included in the selected ID snapshot and current allowed selection policy
4. Property	Every path is editable for the target element type
5. Scope	Every patch writes only to the requested base or viewport override
6. Revision	Recorded base and viewport revisions still match
7. Value	Colors, sizes, URLs, text length, and numeric limits are safe
8. Structure	No self-parent, missing parent, page-root deletion, or ancestor cycle

Invalid means no mutation

Unknown IDs, unselected AI targets, forbidden fields, invalid JSON, malformed payloads, structural cycles, and stale revisions must return an error before current state or history changes.

3.3 Command execution pipeline



```

executeEditCommand(command):
  1. commandSchema.safeParse(command)
  2. validate targets, authority, fields, scope, revisions, and structure
  3. calculate one before/after snapshot for each target
  4. dispatch one commitValidatedCommand action
  5. reducers update template and append history in the same dispatch
  6. persistence middleware saves durable state
  
```

3.4 Commit boundaries

Interaction	One committed step occurs on
Inline text	Enter or blur
Inspector field	Apply or intentional blur
Resize	Pointer release
Drag/reorder	Drop
Code editor	Apply Code button
AI proposal	Accept on one proposal item
Restore	Confirm Restore button

Draft values may update local component state continuously. Only the final intentional boundary creates a command and history entry. This prevents one history item per keystroke or pointer movement.

3.5 History model

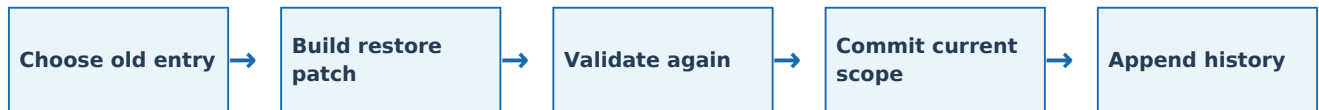
```

interface HistoryEntry {
  id: string;
  commandId: string;
  elementId: string;
  scope: EditScope;
  source: EditSource;
  description: string;
  before: DeepPartial<ElementValues>;
  after: DeepPartial<ElementValues>;
  previousRevision: number;
  resultingRevision: number;
  timestamp: string;
}

type HistoryState = Record<
  string,
  Record<EditScope, HistoryEntry[]>
>;
  
```

- A group edit creates one entry for every affected element, not one inseparable global history record.
- A base history entry is stored under All; desktop/tablet/mobile entries remain separate.
- Every accepted AI change, manual commit, code commit, and restore creates history.

3.6 Independent restoration policy



Restoration is not a time-machine rewind. The user chooses one historical snapshot for one element and one scope. The application calculates a patch from the current scoped values to that snapshot and submits a normal source=restore command. Unrelated elements, viewports, selections, and newer history entries remain untouched.

Action	Required result
Restore services.card.1 / mobile	Only that card's mobile override changes
Restore hero.heading / all	Only that heading's base values change; its existing viewport overrides remain
Restore one accepted AI result	Other elements from the same proposal batch keep their current status and values

Step 3 completion checklist

- Canvas, code, AI acceptance, and restore all call the same executor.
- An invalid command produces an error and no current-state or history mutation.
- Stale checks use base plus relevant viewport revision tokens.
- Every committed target receives a separate scoped history entry.
- Restore creates a new history entry and never deletes newer history.

STEP 4

Canvas selection and high-degree manual editing

Objective: Build a credible visual editing loop over the canonical model without making the DOM a second source of truth.

Deliverable: Keyboard-operable single/group selection, visible overlays, inline editing, design inspector, resizing, reordering, and structure controls.

4.1 Selection state

```
interface EditorUIState {
  selectedIds: string[];
  activeViewport: "desktop" | "tablet" | "mobile";
  editScope: "all" | "desktop" | "tablet" | "mobile";
  activePanel: "design" | "ai" | "code" | "history";
}
```

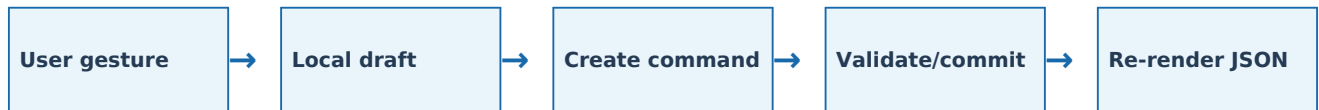
Input	Behavior
Click	Replace selection with the clicked stable ID
Shift/Ctrl/Cmd-click	Toggle the clicked ID in the group
Escape	Clear selection
Enter on focused element	Select the focused element
Space with modifier	Add/remove the focused element from the group
Layers panel	Mirrors canvas selection using the same selectedIds state

- Selected elements receive a visible outline and label; group members remain independent targets.
- Canvas elements need accessible names, visible focus, and keyboard interaction.
- Click handlers call stopPropagation so selecting a child does not accidentally replace it with its parent.

4.2 Manual editing capabilities

Boundary	Controls
Content	Inline text, button label/link, image source/alt text
Style	Text/background color, typography, alignment, spacing, border, radius
Size	Width, height, min/max limits, resize handles and numeric inspector
Position	Scoped X/Y offset or movement within the parent layout
Order	Move up/down and drag reorder with keyboard alternative
Structure	Add, duplicate, move to parent, and soft hide/remove

4.3 Canvas event flow



For contenteditable text, keep a local draft during typing. On Enter or blur, compare the draft with the resolved value and create a command only when it changed. After a successful commit, render the new canonical value. If validation fails, show an error and restore the last valid resolved value.

4.4 Design inspector behavior

- When one element is selected, display every allowed property for its type.
- When multiple same-type elements are selected, display shared values or a Mixed state.
- A group Apply generates one target patch per selected element, preserving independent history.
- Always display the current edit scope near the Apply action.
- Show which viewports will change and which are protected.

4.5 Additional product capability: Viewport Impact Indicator

Desktop	Affected
Tablet	Protected
Mobile	Protected

User problem: a non-technical editor can forget the selected scope and accidentally change another screen size. The indicator makes scope consequences visible before commit. Validate it by measuring wrong-viewport edits, immediate restores, task completion, and whether users can correctly predict affected views.

Step 4 completion checklist

- Single and modifier-based group selection work in canvas and layers panel.
- Selection and focus are visible and keyboard-operable.
- Manual controls cover content, style, size, position, order, and structure.
- Every manual commit updates canonical state and creates scoped history.
- The viewport impact indicator clearly distinguishes affected and protected views.

STEP 5

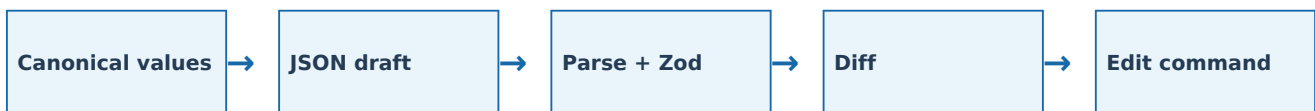
Validated code editing and deterministic AI proposal workflow

Objective: Keep code and canvas synchronized while making generated edits selection-bound, reviewable, and independently applicable.

Deliverable: A safe JSON code surface, five deterministic AI paths, visible failure behavior, before/after review, partial acceptance, and stale handling.

5.1 JSON code surface

Use JSON rather than arbitrary JSX or HTML. The assignment's durable model is typed and JSON-serializable, so a selected-component JSON surface demonstrates high-degree code editing without introducing a second parser or unsafe raw markup.



```

{
  "content": { "text": "Grow your business" },
  "style": {
    "fontSize": 48,
    "color": "#172033",
    "textAlign": "left"
  },
  "layout": {
    "width": 70,
    "order": 1,
    "visible": true
  }
}
  
```

Code Apply rules

- The editor displays only editable values for the chosen element and scope.
- The user cannot edit ID, element type, revision counters, history, or proposal status.
- Apply parses JSON, validates it with the element-type schema, calculates a typed diff, and calls the normal command executor.
- Invalid JSON or invalid fields show an error and leave the last valid canvas state unchanged.
- The invalid text remains as a draft so the user can correct it.

5.2 Deterministic AI engine contract

```

generateProposals({
  instruction,
  selectedIds,
  scope,
  template
}): ProposalBatch
  
```

Rules:

- Pure function; no dispatch and no mutation
- Normalize text and match documented scenarios
- Use current selected IDs, types, values, and revision tokens
- No randomness, timestamps, or network calls in scenario output
- Same instruction + selection + state = same proposal

5.3 Required reviewer-visible scenarios

Path	Example instruction	Deterministic result
Content rewrite	Make this text more concise	Shorten each selected text element's current content deterministically
Style	Make the selected element dark blue	Propose allowed background and text colors based on element type
Move/resize/reorder	Make this card wider and move it first	Calculate width/order changes from current values
One viewport	Stack selected items vertically	Require a single viewport scope; change eligible container direction only there
Multi-element	Make the selected cards compact	Reduce current padding per selected eligible card and return independent items
Safe failure	Add a payment system	Return unsupported with no proposal and no state change

Not a fixed replacement

Each scenario must inspect the current selected IDs, element types, values, viewport scope, and revisions. It cannot replace the whole page with one predetermined template.

5.4 Proposal model

```
interface ProposalItem {
  id: string;
  elementId: string;
  scope: EditScope;
  before: DeepPartial<ElementValues>;
  after: DeepPartial<ElementValues>;
  changes: PropertyChange[];
  revisionToken: RevisionToken;
  status: "pending" | "accepted" | "rejected" | "invalid" | "stale";
  error?: string;
}

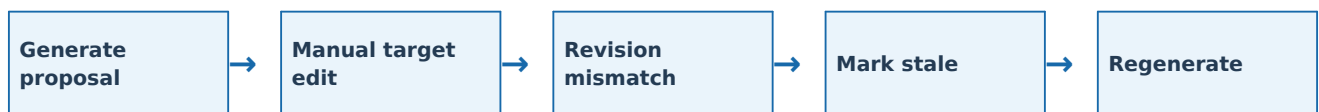
interface ProposalBatch {
  id: string;
  instruction: string;
  selectedIdsSnapshot: string[];
  scope: EditScope;
  items: ProposalItem[];
}
```


5.5 Proposal review and partial acceptance

Element	Before/after	Independent status
Service Card 1	Padding 24 -> 16	Accepted
Service Card 2	Padding 24 -> 16	Rejected
Service Card 3	Padding 20 -> 12	Pending

Accepting one proposal item creates a single-target source=ai command. Rejecting an item only changes that proposal status. Accept All may loop through items, but each item must be revalidated and committed independently so one invalid or stale target does not force the same outcome on the others.

5.6 Stale proposal behavior



- Before Accept, compare the recorded revision token with the current element token.
- If it differs, do not apply the old values and mark only that item stale.
- Explain that the element changed after proposal generation and offer regeneration.
- An accepted item creates normal scoped history and is later independently restorable.

Step 5 completion checklist

- Valid JSON edits update the same state and renderer as canvas edits.
- Invalid JSON cannot damage the last valid canonical state.
- Five documented deterministic paths and at least one safe failure are visible to reviewers.
- AI output targets only selected IDs and allowed fields at the requested scope.
- Before/after review, independent acceptance/rejection, invalid, and stale statuses work.

STEP 6

Persistence, accessibility, responsive quality, and verification

Objective: Prove that the safe architecture survives refresh, works with a keyboard, fits required widths, and is protected by focused tests.

Deliverable: Versioned local persistence, deliberate reset, accessible editor controls, automated safety tests, and a manual QA matrix.

6.1 Persistence contract

```
const PERSISTENCE_KEY = "scoped-ai-editor:v1";
```

```
interface PersistedEditorState {
  schemaVersion: number;
  template: TemplateState;
  history: HistoryState;
}
```

Startup:

```
localStorage.getItem -> JSON.parse -> Zod validate
valid -> hydrate template + history
invalid -> load original seed template and show recovery notice
```

- Save only after a validated commit, not on every draft keystroke.
- Schema version allows future migration or deliberate fallback when saved data becomes incompatible.
- Reset requires confirmation, clears durable state, reloads the seed template, and explains that revisions will be removed.

6.2 Keyboard and accessibility requirements

Area	Required behavior
Selection	Focusable elements, Enter to select, modifier-assisted group selection, Escape to clear
Viewport	Real labeled buttons or tabs; visible active state and focus
Scope	Labeled select/radio group; affected views stated in text
Inspector	Every form control has a visible label and validation message
Reordering	Move Up/Down buttons or dnd-kit keyboard sensor as an alternative to dragging
Proposals	Accept/Reject buttons per item; status announced in text, not color alone
History	Focusable entries and explicit Restore button with scope in the label
Focus	Never remove the browser focus outline without an equally visible replacement

6.3 Focused automated test matrix

Test area	Assertion
AI selection	Proposal/command targeting an unselected ID is rejected with no mutation
AI field boundary	Forbidden property for the selected element type is rejected
AI viewport	Proposal cannot write outside the requested scope
Safe failure	Unsupported instruction produces no proposal and no state change
View isolation	Mobile-only edit leaves desktop and tablet state unchanged
Shared values	Base edit preserves an existing mobile override
Stale revision	Proposal generated before a target edit cannot be accepted afterward
Invalid code	Malformed/invalid JSON leaves canonical state unchanged
State consistency	Equivalent canvas and code commands produce the same canonical result
Partial acceptance	Accepting one item does not accept or mutate its peers
Recovery	Restoring one element does not change another element
Scoped recovery	Restoring mobile history does not change desktop or tablet overrides
Persistence	Refresh rehydrates current template and scoped history

6.4 Suggested commands

```

npm run typecheck
npm run lint
npm run test -- --run
npx playwright test
npm run build

```

6.5 Manual responsive QA

Scenario	Pass condition
Editor at 1280	Toolbar, sidebars, canvas, proposals, and controls remain usable
Desktop preview 1440	Scaled without template clipping; overlays remain aligned
Tablet preview 768	Template resolves expected tablet values with no accidental overflow
Mobile preview 375	No horizontal overflow; text and controls remain legible
Keyboard-only	Complete select, edit, scope, proposal, and restore journey without pointer
Refresh	Current values and history return; drafts/proposals may reset according to documentation

Step 6 completion checklist

- Saved state is validated before hydration and intentionally resettable.
- All required journeys work with keyboard and visible focus.
- The editor is usable at 1280 px and previews approximate 1440/768/375 px safely.
- Focused tests cover AI scope, state consistency, view isolation, and independent recovery.
- Typecheck, lint, unit tests, end-to-end tests, and production build pass.

STEP 7

Documentation, deployment, reviewer demo, and submission

Objective: Convert the completed implementation into clear evidence that maps directly to every assignment requirement.

Deliverable: Meaningful commits, complete README/PRODUCT_NOTES/AI_USAGE, deployed product, optional recording, and final submission checklist.

7.1 README.md contents

Section	Evidence
Overview	Primary user, problem, safe editing goal, screenshots or short GIF if useful
Setup	Node requirement, install, dev, test, build, and preview commands
Template source	State that the template is original; credit any external assets separately
Demo examples	List exact deterministic instructions, required selection/type/scope, and expected result
Architecture	Canonical model, command pipeline, resolver, history, persistence, and component boundaries
Resolution rule	Base values plus viewport override; existing overrides win over later shared changes
Commit boundary	Explain when drafts become commands and why history is not created per keystroke
Trade-off	JSON code editor is safer and model-native but does not support arbitrary JSX/HTML
Libraries	Identify each editor/component library and what the application still owns
Requirement map	Assignment requirement -> component/module/test evidence
Limitations	Honest cuts, assumptions, known uncertainty, and prioritized next steps

7.2 PRODUCT_NOTES.md contents

- Primary user, their job, and the definition of a safe completed template edit.
- Definition of an element, group selection, committed step, viewport scope, and editable-property boundary.
- How canvas and code editing share canonical state and the same command pipeline.
- How base values resolve with desktop, tablet, and mobile overrides.
- How deterministic paths remain inside selection and viewport scope.
- How invalid payloads and stale revisions are handled without mutation.
- Before/after review, partial acceptance, and per-element/per-viewport recovery policy.
- Viewport Impact Indicator: user problem, reason, and evidence that would validate it.
- Cuts, assumptions, and the next three improvements in priority order.

7.3 AI_USAGE.md contents

Evidence	What to record
Tools/models	Name each tool and the bounded task it helped with
Planning example	Short redacted interaction about architecture or product framing
Implementation example	Short redacted debugging, validation, or testing interaction
Rejected/corrected suggestion	Describe one weak suggestion, why it was unsafe, and the resulting correction
Verification	Commands, automated tests, manual scenarios, dependency review, and remaining uncertainty
Workflow limitation	What the AI workflow missed and what you would change next time

Privacy

Redact API keys, employer/client material, private repository content, and unrelated personal prompts.

You remain responsible for every submitted line and dependency.

7.4 Deployment

Before deployment:

```
npm run typecheck
npm run lint
npm run test -- --run
npm run build
```

Vercel:

```
Framework preset: Vite
Build command: npm run build
Output directory: dist
```

After deployment:

Repeat desktop/tablet/mobile, refresh, reset, proposal, and restore journeys.

7.5 Final reviewer demonstration

Order	Reviewer action
1	Open the original responsive template.
2	Switch through desktop, tablet, and mobile previews.
3	Select and manually edit the hero heading.
4	Edit the same component through validated JSON.
5	Apply a mobile-only change and prove desktop/tablet remain unchanged.
6	Select three service cards as a group.
7	Run: Make the selected cards compact.
8	Accept one card, reject one card, and leave one pending.
9	Run the unsupported Add a payment system instruction and show safe failure.
10	Generate a proposal, manually change the target, and show stale rejection.
11	Open one element's history filtered to a viewport scope.
12	Restore an older mobile revision and prove unrelated work remains.
13	Refresh and show that current template plus history persist.
14	Use the deliberate reset action with confirmation.

7.6 Submission package

Deliverable	Required evidence
Repository	Accessible Git repository, meaningful incremental commits, and no secrets
Working product	Deployed Vercel link preferred; otherwise exact local setup plus short recording
README.md	Setup, source, examples, architecture, trade-off, tests, and requirement mapping
AI_USAGE.md	All five evidence areas, redacted where necessary
PRODUCT_NOTES.md	User, safety definitions, scope, recovery, added capability, cuts, priorities

Step 7 completion checklist

- All three documentation files match the actual implementation rather than planned behavior.
- Repository history contains meaningful feature-sized commits and no secrets.
- Production deployment passes the same tests and manual journeys as local development.
- The fourteen-step reviewer demo can be completed without explanation-dependent workarounds.
- Known cuts and the next three improvements are explicit and credible.

Build schedule and dependency order

Time	Step	Primary work
Day 1	Step 1	Scaffold, architecture, Redux boundaries, docs skeleton, original template plan
Day 2	Step 2	Types, schemas, seed JSON, resolver, renderer, device previews
Day 3	Step 3	Commands, validation, revisions, commits, history, restore tests
Day 4	Step 4	Selection, layers, inspector, inline editing, reorder/resize/structure
Day 5	Step 5	CodeMirror JSON surface, deterministic scenarios, proposals, partial acceptance
Day 6	Step 6	Persistence, reset, keyboard/accessibility, unit and end-to-end tests
Day 7	Step 7	Documentation, responsive polish, deployment, complete reviewer rehearsal
Days 8-10	Buffer	Fix difficult interaction, stale, responsive, or accessibility defects; record demo

Correct starting point

Begin with `src/model/types.ts`, `src/model/schemas.ts`, `src/model/seedTemplate.ts`, `src/model/resolveElement.ts`, and `src/model/resolveChildren.ts`. Do not begin with the AI panel or drag-and-drop. The first milestone is a complete responsive template rendered entirely from validated JSON.

First implementation milestone

- The seed template contains approximately 15-25 modular elements with stable IDs.
- Element type, content, style, layout, base, viewport overrides, and revisions are typed and runtime-validated.
- TemplateRenderer recursively renders the normalized model through normal React components.
- Desktop, tablet, and mobile resolution is explicit and proven by tests.
- No editing exists until this foundation is stable.

Overall definition of done

Area	Complete when
Model	Typed, serializable, stable IDs, base plus overrides, revisions, and scoped history
Editing	Content, style, position, size, order, and structure through canvas/code
Selection	Visible, keyboard-operable, single and group targets stored as stable IDs
Responsive	All/Desktop/Tablet/Mobile isolation with clear resolution rule
AI demo	Deterministic typed proposals restricted to selection, fields, scope, and revision
Review	Before/after per element with pending/accepted/rejected/invalid/stale states
Recovery	Per element and scope, independent, and recorded as a new history entry
Persistence	Validated current template/history survive refresh; deliberate reset works
Quality	1280 editor, 1440/768/375 previews, keyboard support, labels, visible focus
Evidence	Focused tests, meaningful commits, deployed link, README, AI_USAGE, PRODUCT_NOTES

Architecture rules to protect throughout development

1. The React DOM is a rendering result, never the durable source of truth.
2. Every selected target is a stable element ID.
3. Canvas, code, accepted AI, and restore actions share the same command and validation pipeline.
4. A single-view edit writes only to that viewport override.
5. Existing viewport overrides win over later base/shared changes.
6. AI output is a proposal and cannot mutate current state before approval.
7. Every proposal item is independently acceptable, rejectable, invalid, stale, and restorable.
8. Invalid or stale input produces no current-state or history mutation.
9. Every committed target creates element-and-scope history.
10. Restore is a new commit, not a destructive rewind.
11. Only validated durable state is persisted.
12. Accessibility and keyboard alternatives are product behavior, not final styling tasks.
13. Documentation must explain the actual implementation boundaries and trade-offs.

Final recommendation

Treat Steps 2 and 3 as the engineering core. A visually polished editor with weak scope validation or global history will miss the assignment's central goal. A clean, modest UI over a strong canonical model, command pipeline, and independent recovery policy will produce much stronger evidence.

End of architecture and implementation plan